

Pagination and Sorting with an Employee Directory API

The team is struggling with the current employee directory — every request dumps the entire dataset in one response with no way to control order or page size. With the company growing, this will not scale. All the basic endpoints are already working. Your job is to progressively improve the API by introducing pagination, sorting, and department filtering so the frontend team gets exactly the data they need, in the order they need it, one page at a time.

What's already provided:

- `Employee.java` — entity with `id`, `firstName`, `lastName`, `department`, `salary`
- `EmployeeRepository.java` — `JpaRepository`, no custom methods yet
- `EmployeeService.java` — `getAllEmployees()` returns a flat `List<Employee>`
- `EmployeeController.java` — `GET /api/employees` returns all 25 employees at once
- `DataLoader.java` — seeds 25 employees across 5 departments on startup
- `application.properties` — H2 in-memory database, ready to use

Specifically, you are expected to:

1. Hit the existing endpoint and experience the problem firsthand — all 25 records returned at once with no structure or ordering.
2. Refactor the service and controller to use `Pageable` and return `Page<Employee>` instead of `List<Employee>`, so callers can request a specific page and size.
3. Extend the endpoint to accept a `sort` parameter so callers can order results by any field in ascending or descending order.
4. Add an optional `?department=` filter that works alongside pagination and sorting so the HR team can browse employees from a specific department.
5. Explore multi-field sorting so results are ordered by department first, then by last name within each department.

Deliverables:

- `GET /api/employees` without params returns the first page with default size and sort.
- `GET /api/employees?page=0&size=5` returns exactly 5 employees with page metadata.

- `GET /api/employees?sort=lastName,asc` returns all employees sorted alphabetically.
- `GET /api/employees?department=Engineering&page=0&size=3` returns paginated results for that department only.
- `GET /api/employees?sort=department,asc&sort=lastName,asc` returns employees sorted by department then name.
- Response includes `totalElements`, `totalPages`, `first`, and `last` metadata fields.

Testing Endpoints

Experience the problem first:

```
curl -s http://localhost:8080/api/employees | cat
```

Basic pagination:

```
curl -s "http://localhost:8080/api/employees?page=0&size=5" | cat
curl -s "http://localhost:8080/api/employees?page=1&size=5" | cat
curl -s "http://localhost:8080/api/employees?page=4&size=5" | cat
```

Sort by last name ascending:

```
curl -s "http://localhost:8080/api/employees?sort=lastName,asc" | cat
```

Sort by last name descending:

```
curl -s "http://localhost:8080/api/employees?sort=lastName,desc" | cat
```

Sort by salary descending:

```
curl -s "http://localhost:8080/api/employees?sort=salary,desc" | cat
```

Sort by department:

```
curl -s "http://localhost:8080/api/employees?sort=department,asc" | cat
```

Pagination and sorting combined:

```
curl -s "http://localhost:8080/api/employees?page=0&size=5&sort=lastName,asc" |
cat
```

Filter by department:

```
curl -s "http://localhost:8080/api/employees?department=Engineering" | cat
```

Filter by department with sorting:

```
curl -s "http://localhost:8080/api/employees?department=Legal&sort=salary,desc"  
| cat
```

Filter by department with pagination:

```
curl -s "http://localhost:8080/api/employees?department=HR&page=0&size=3" | cat
```

Sort by multiple fields:

```
curl -s  
"http://localhost:8080/api/employees?sort=department,asc&sort=lastName,asc" |  
cat
```